SESSION 2

# Core Techniques & Task Patterns

2

The everyday toolkit, and how prompting changes with the task.

Mercredi 16 septembre · 8h00–9h40 · 1h40

Prompt & Context Engineering — Stefan Duprey

TODAY

# Session 2 — plan & goals

**You will be able to**

- Branch, merge, and resolve a conflict in Git
- Apply zero/few-shot, role & structure
- Match prompt patterns to task types
- Diagnose and fix weak prompts

RUN OF SHOW

| | |
|---|---|
| 0:00 | Git — branching, merging, conflicts |
| 0:55 | Techniques & task patterns |
| 1:25 | Lab — fix prompts, build a classifier |

# Branching & merging

1

## Branch

git switch -c feature — isolate your work from main.

2

## Work

Commit freely on the branch; main stays clean.

3

## Merge

git merge — bring the finished work back into main.

4

## Delete

Remove the branch once it's merged.

# Resolving a merge conflict

- Conflicts happen when two branches change the same lines
- Git marks the clash directly in the file
- You decide the correct result and remove the markers
- Then stage and commit the resolution

conflict markers

```
<<<<<<< HEAD
temperature = 0.0
=======
temperature = 0.7
>>>>>>> feature

# edit to the right value, then:
git add config.py
git commit
```

OVERVIEW

# Four levers you control

Almost everything you do to a prompt is a turn of one of these four dials — keep them in mind all course.

### I Instruction

Say exactly what to do, for whom, and to what standard. Most weak outputs are really under-specified instructions.

### E Examples

Show the pattern instead of describing it. A few input→output pairs often beat a paragraph of rules.

### R Role

A persona sets vocabulary, depth and default rigor — 'you are a careful editor' changes tone reliably.

### F Format

Constrain the shape of the output: length, a schema, a style. Formats you can check are formats you can trust.

TECHNIQUE

# Zero-shot: just ask, but ask well

The simplest move is to just ask — the trick is asking precisely enough that there's only one reasonable answer.

- **No examples — a precise instruction**

You rely entirely on the wording. Works when the task is common and unambiguous.

- **Name the audience and format**

Vague asks get vague answers. 'One lowercase word' removes a whole class of formatting problems.

- **The right default**

Start here. Add examples only when zero-shot visibly falls short — don't pay for examples you don't need.

```
zero-shot

System:
 You are a support triage bot.

User: Classify the ticket as
billing, bug, or other.
Reply with one lowercase word.

Ticket: "App crashes on export."
```

TECHNIQUE

# Few-shot: show the pattern

When words alone aren't landing, stop explaining and start showing — a few examples often do what a paragraph can't.

- **Give 2–5 input→output examples**

The model infers the task and the exact output shape from your examples — often more reliably than from instructions alone.

- **Consistency beats quantity**

Keep the format identical across examples. Order can matter; a contradictory example poisons the batch.

- **Essential on smaller models**

Local models drift without examples. Few-shot is how you get reliable structure out of a modest model (revisited in S7).

few-shot

```
"Refund took weeks"  -> billing
"Button does nothing" -> bug
"Love the new UI"    -> other

"Charged twice"      -> ?
```

TECHNIQUE

# Role & structure

Two quiet levers that shape a lot: who you tell the model to be, and how you fence off the parts of your prompt.

- **A persona shifts the whole register**

'You are a senior tax accountant' changes vocabulary, caution and defaults. It steers style — it does not add knowledge the model lacks.

- **Delimiters fence the input**

Wrap user text in quotes or tags so the model treats it as data, not as new instructions. This is also your first line against injection (S8).

- **Label the sections**

Context / Task / Constraints / Output headings make long prompts legible to the model and to you.

WATCH OUT

A role reliably changes how the model writes. It does not make the model know things it was never trained on — don't confuse tone for expertise.

PATTERNS

# Prompts differ by task

There's no single 'good prompt' — the right shape depends on what kind of job you're asking for.

### # Classification

Fixed label set, one-word output, few-shot examples. Optimise for a parseable, constrained answer.

### ↘ Extraction

Pull typed fields into a schema. Delimit the source text and specify what to do when a field is missing.

### ¶ Summarization

Decide audience and length first, then what to keep versus drop. 'Summarize' without those is under-specified.

### ✎ Generation

Open-ended writing. Supply style, hard constraints, and one example of 'good' to anchor quality.

DIAGNOSIS

# Common failure modes

When a prompt misbehaves it's usually one of these four — think of this as your debugging checklist.

- **Ambiguity → the model guesses**

If the instruction allows two readings, the model will sometimes pick the wrong one. Remove the ambiguity, don't add pleading.

- **Format drift → unparseable output**

You asked for JSON and got prose with an apology. Constrain the format and, later, validate it (S4).

- **Over-stuffed context → lost instruction**

The key line drowns in pasted text. Shorten, or move it to the edges of the prompt.

- **Leaked instructions → input obeyed**

Unfenced user text gets read as a command. Delimiter discipline matters (and it's a security issue — S8).

FIRST FIX

Most bad outputs are under-specified prompts, not weak models. Tighten the ask before you reach for a bigger model.

HANDS-ON LAB

# Lab — Fix prompts & build a classifier

LOCAL · OLLAMA

`prompt-engineering-course/02_techniques`

## TASKS

1. Create a branch, then rewrite three weak prompts to spec
2. Build a few-shot sentiment classifier with a fixed label set
3. Add and remove examples; watch the behaviour change
4. Commit each version with a message saying what you changed and why

## DELIVERABLE

A before/after prompt set and a working classifier, committed step by step so the Git history shows your iteration.

WRAP-UP

# Recap — Session 2

- ✓ Instruction, examples, role, format are your four levers
- ✓ Few-shot fixes output format and the label space
- ✓ Different task types want different prompt shapes
- ✓ Tighten the ask before blaming the model

NEXT SESSION

## Evaluation & iteration

The pivot: turn prompting from vibes into engineering by measuring. Eval sets, LLM-as-judge, and prompt sensitivity.